RAG Demo using PDF + Chroma

This notebook demonstrates Retrieval-Augmented Generation (RAG) using a PDF document, Chroma vector database, and optional Gemini LLM.

```
# Import necessary libraries from langchain_community for document loading, embeddings, and vector stores
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma
import os
```

```
# Install required Python packages silently (`-q`).
# langchain: Core LangChain library.
# langchain-community: Community contributed LangChain components (loaders, embeddings, vector stores).
# chromadb: The Chroma vector database.
# pypdf: PDF document parsing library.
# sentence-transformers: For generating embeddings.
# google-generativeai: For integrating with Google's Generative AI models.
# langchain-text-splitters: For text splitting functionalities in LangChain.
!pip -q install langchain langchain-community chromadb pypdf sentence-transformers google-generativeai la
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 52.0/52.0 kB 5.7 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.5/2.5 MB 59.0 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 21.1/21.1 MB 90.5 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 329.1/329.1 kB 34.7 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 278.2/278.2 kB 28.3 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.0/2.0 MB 102.0 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.0/1.0 MB 71.0 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 17.1/17.1 MB 99.4 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 72.5/72.5 kB 8.2 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 132.6/132.6 kB 13.1 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 66.4/66.4 kB 8.2 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 220.0/220.0 kB 25.3 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 105.4/105.4 kB 12.6 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 71.6/71.6 kB 8.0 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.6/60.6 kB 6.4 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 64.7/64.7 kB 7.5 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 51.0/51.0 kB 6.0 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are installe
google-colab 1.0.0 requires requests==2.32.4, but you have requests 2.32.5 which is incompatible.
opentelemetry-exporter-otlp-proto-http 1.37.0 requires opentelemetry-exporter-otlp-proto-common==1.37.0,
opentelemetry-exporter-otlp-proto-http 1.37.0 requires opentelemetry-proto==1.37.0, but you have opentel
opentelemetry-exporter-otlp-proto-http 1.37.0 requires opentelemetry-sdk~=1.37.0, but you have opentelem
opentelemetry-exporter-gcp-logging 1.11.0a0 requires opentelemetry-sdk<1.39.0,>=1.35.0, but you have ope
google-adk 1.23.0 requires opentelemetry-api<=1.37.0,>=1.37.0, but you have opentelemetry-api 1.39.1 whi
google-adk 1.23.0 requires opentelemetry-sdk<=1.37.0,>=1.37.0, but you have opentelemetry-sdk 1.39.1 whi
```

```
# Define the path to the PDF document.
PDF_PATH = "/content/Promotional Content Creation.pdf"
# Initialize PyPDFLoader with the PDF path to load the document.
loader = PyPDFLoader(PDF_PATH)
# Load the pages from the PDF, resulting in a list of Document objects.
docs = loader.load()
# Print the number of pages successfully loaded from the PDF.
print("Pages loaded:", len(docs))
```

```
Pages loaded: 7
```

```
# Initialize a RecursiveCharacterTextSplitter to break down documents into smaller chunks.
# chunk_size=900: Specifies the maximum size of each text chunk.
# chunk_overlap=150: Defines the number of characters that consecutive chunks will overlap,
#                    helping to maintain context across splits.
text_splitter = RecursiveCharacterTextSplitter(chunk_size=900, chunk_overlap=150)
# Split the loaded documents into smaller, manageable chunks.
chunks = text_splitter.split_documents(docs)
# Print the total number of chunks created.
print("Total chunks:", len(chunks))
```

```
Total chunks: 25
```

```
# Initialize HuggingFaceEmbeddings to convert text into numerical vectors.
# model_name="sentence-transformers/all-MiniLM-L6-v2": Specifies the pre-trained model to use for embed
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
# Define the directory where the Chroma vector database will be persisted.
persist_dir = "/mnt/data/chroma_kncet"
# Create a Chroma vector database from the text chunks and embeddings.
# The database will be stored in the specified `persist_directory`.
vectordb = Chroma.from_documents(chunks, embeddings, persist_directory=persist_dir)
# Persist the vector database to disk (note: in newer Chroma versions, this might be automatic).
vectordb.persist()
# Confirm that the vector database has been created.
print("Vector DB created")
```

```
/tmp/ipython-input-3888972587.py:3: LangChainDeprecationWarning: The class `HuggingFaceEmbeddings` was d
  embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/s
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Ple
```

modules.json: 100% 349/349 [00:00<00:00, 38.0kB/s]

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 13.7kB/s]

README.md: 10.5k/? [00:00<00:00, 912kB/s]

sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 5.88kB/s]

config.json: 100% 612/612 [00:00<00:00, 64.4kB/s]

model.safetensors: 100% 90.9M/90.9M [00:00<00:00, 121MB/s]

Loading weights: 100% 103/103 [00:00<00:00, 460.80it/s, Materializing param=pooler.dense.weight]

```
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect iden
```

tokenizer_config.json: 100% 350/350 [00:00<00:00, 18.8kB/s]

vocab.txt: 232k/? [00:00<00:00, 10.5MB/s]

tokenizer.json: 466k/? [00:00<00:00, 21.4MB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 13.5kB/s]

config.json: 100% 190/190 [00:00<00:00, 22.0kB/s]

```
Vector DB created
/tmp/ipython-input-3888972587.py:10: LangChainDeprecationWarning: Since Chroma 0.4.x the manual persiste
  vectordb.persist()
```

```
# Initialize the retriever from the Chroma vector database.
# It's configured to search for the top 4 most relevant documents.
retriever = vectordb.as_retriever(search_kwargs={"k":4})

# Define the query string to search for within the documents.
query =     "List 3 key reasons to choose KNCET."

# Example queries (commented out) demonstrating other possible searches:
# "What are the main accreditations and autonomous status details mentioned?"
# "What infrastructure highlights are mentioned (campus size, labs, library, Wi-Fi)?",
# "What placement details are mentioned (highest and average package, recruiters)?",

# Execute the query using the retriever to get relevant documents.
docs = retriever.invoke(query)

# Iterate through each retrieved document.
for d in docs:
    # Clean the document content by removing null characters (\x00) which can appear from PDF processing.
    cleaned_content = d.page_content.replace('\x00', '') # Remove null characters
    # Print the cleaned content, prefixed with a bullet point and truncated to the first 400 characters t
    print(f"* {cleaned_content[:400]}")
```

```
* Why Choose KNCET?
NAAC A+ & NBA accredited programmes – quality engineering education.
Autonomous curriculum with flexibility to introduce modern industry-relevant courses.
Eco
†friendly campus with smart classrooms, digital library and 100+ labs.
Scholarships & fee concessions for meritorious and sports students from disadvantaged
backgrounds.
Support for research & start
†ups via the Kongunadu I
* From Campus to Corporate )ö – Our Campus-to-Corporate centre trains you for success and
connects you with top recruiters like TCS, Wipro & HCL. Begin your journey with us today!
#Placements #CareerReady
Hands
†on Learning )÷ – With 100+ laboratories and a state
†of
†the
†art digital library, KNCET
gives you the tools to innovate. Join a college where experiments become in
* Engineering & Technology (KNCET) – 70-acre green campus, NAAC A+ & NBA accreditations and
strong placements (highest package ₹12 LPA). Admissions open! Reply YES to know more.
Hi! Start your engineering journey in a college that offers smart classrooms, 100+ labs, digital library
```